

▼ Módulo 2: Operadores

Certifique-se de que você está dentro da pasta da aula antes de executar o comando abaixo. O comando criará automaticamente as subpastas `cpp/src`, `cpp/bin` e `cpp/lib` dentro dela. Caso tenha executado anteriormente, não é necessário reexecutar.

```
!mkdir -p cpp/src cpp/bin cpp/lib
```

A estrutura gerada será a seguinte.

```
NOME_DA_AULA/
├── cpp/
│   ├── src/
│   ├── bin/
│   └── lib/
```

Pasta	Conteúdo	Extensões
<code>src</code>	Arquivos fonte com a lógica do programa	<code>.cpp</code>
<code>bin</code>	Executáveis gerados pelo compilador	sem extensão
<code>lib</code>	Cabeçalhos e bibliotecas externas	<code>.h</code> <code>.hpp</code>

Nota sobre os comandos do Jupyter Notebook

Você vai notar que os exemplos desta aula utilizam alguns comandos especiais, como `%%writefile`, `!g++` e `!./...`. É importante entender que **esses prefixos não fazem parte do C++**, O `%%writefile` e o `!` são apenas uma "ponte" entre o Jupyter e o terminal do sistema. **O fluxo real é o mesmo que você vê nos vídeos:** escrever o código, compilar com g++ e executar o binário. — eles são recursos do próprio Jupyter Notebook que nos permitem escrever e executar código C++ diretamente por aqui.

- `%%writefile cpp/src/02_exec_02.cpp` salva o conteúdo da célula como um arquivo `.cpp` no disco.
- `!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02` compila o arquivo usando o g++, exatamente como você faria no terminal.
- `!./cpp/bin/02_exec_02` executa o binário gerado.

Disponibilizamos os exemplos dessa forma para que você consiga acompanhar e rodar tudo diretamente na aula, sem precisar sair do notebook, de modo online e interativo. Mas recomendamos que, sempre que possível, **pratique também pelo terminal**, como demonstrado nos vídeos — é assim que você vai trabalhar no dia a dia com C++.

▼ 2.1. Atribuição e Operadores Aritméticos

O operador de atribuição `=` copia o valor que está do lado direito para a variável que está do lado esquerdo. Esse é o operador mais fundamental da linguagem, pois toda variável precisa receber um valor antes de ser utilizada.

```
int result = 10; // o valor 10 é copiado para a variável result
```

Os operadores aritméticos permitem realizar as operações matemáticas básicas sobre variáveis numéricas. A tabela abaixo resume cada um deles.

Operador	Operação	Exemplo	Resultado
<code>+</code>	Adição	<code>2 + 7</code>	<code>9</code>
<code>-</code>	Subtração	<code>2 - 7</code>	<code>-5</code>
<code>*</code>	Multiplicação	<code>2 * 7</code>	<code>14</code>
<code>/</code>	Divisão	<code>7 / 2</code>	<code>3</code>
<code>%</code>	Módulo (resto)	<code>7 % 2</code>	<code>1</code>

Os quatro primeiros operadores seguem o comportamento esperado da matemática. O operador `%`, chamado de módulo, retorna o resto da divisão inteira entre dois valores. Por exemplo, `31 % 10` resulta em `1`, pois 31 dividido por 10 é igual a 3 com resto 1.

Um ponto que merece atenção especial é a divisão entre inteiros. Quando os dois operandos são do tipo `int`, o C++ descarta a parte fracionária do resultado. Observe o trecho abaixo.

```
int number1 {2};
int number2 {7};
int result = number1 / number2;
// Resultado esperado: 0.285...
// Resultado obtido: 0 (a parte decimal é descartada)
```

O valor armazenado em `result` é `0`, e não `0.285`. Isso acontece porque a divisão entre dois inteiros sempre produz um inteiro, truncando qualquer parte decimal sem arredondamento. Para obter o resultado com casas decimais, pelo menos um dos operandos precisa ser do tipo `float` ou `double`.

```
double result = 2.0 / 7;
// Agora o resultado é 0.285714... pois um dos operandos é double
```

O programa abaixo aplica cada operador aritmético sobre duas variáveis inteiras e imprime os resultados. Acompanhe a saída esperada com base nas regras apresentadas acima, prestando atenção especial ao resultado da divisão.

Exemplo 1: Operações aritméticas básicas

```
%%writefile cpp/src/02_exec_01.cpp

#include <iostream>

using namespace std;

int main(){

    int number1 {2};
    int number2 {7};

    // Adição
    int result = number1 + number2;
    cout << "Adição: " << result << endl;

    // Subtração
    result = number1 - number2;
    cout << "Subtração: " << result << endl;

    // Multiplicação
    result = number1 * number2;
    cout << "Multiplicação: " << result << endl;

    // Divisão
    result = number1 / number2;
    cout << "Divisão: " << result << endl;

    // Modulo
    result = number1 % number2;
    cout << "Módulo: " << result << endl;

    result = 31 % 10;
    cout << "Módulo: " << result << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_01.cpp -o cpp/bin/02_exec_01
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_01`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/02_exec_01
```

Assim como na matemática, o C++ respeita uma ordem de prioridade entre os operadores aritméticos. Multiplicação, divisão e módulo são sempre resolvidos antes de adição e subtração. Quando dois operadores possuem a mesma prioridade, a avaliação ocorre da esquerda para a direita. Para alterar essa ordem natural, utilizamos parênteses `()`.

Prioridade	Operadores	Avaliação
Maior	<code>*</code> <code>/</code> <code>%</code>	Esquerda para direita
Menor	<code>+</code> <code>-</code>	Esquerda para direita
Qualquer	<code>()</code>	Força prioridade

Para entender como o compilador avalia uma expressão, acompanhe os três casos abaixo.

Caso 1. Sem parênteses, o compilador resolve primeiro os operadores de maior prioridade.

```
int result = 3 + 4 * 2 - 10 / 5;
//          4 * 2 = 8      (multiplicação primeiro)
//          10 / 5 = 2      (divisão primeiro)
//          3 + 8 - 2      (agora, esquerda para direita)
//          Resultado: 9
```

Caso 2. Quando operadores de mesma prioridade aparecem na sequência, a avaliação segue da esquerda para a direita.

```
int result = 20 / 4 * 3 + 1;
//          20 / 4 = 5      (divisão, da esquerda para a direita)
//          5 * 3 = 15      (multiplicação, próximo da esquerda)
//          15 + 1          (adição por último)
//          Resultado: 16
```

Perceba que a divisão $20 / 4$ é resolvida antes da multiplicação $* 3$ simplesmente porque aparece mais à esquerda. Não é a divisão que tem prioridade sobre a multiplicação, ambas possuem o mesmo peso. O desempate acontece pela posição na expressão.

Caso 3. Os parênteses forçam a adição a ser calculada antes da multiplicação.

```
int result = (2 + 3) * 4 - 10 / 5;
//          (2 + 3) = 5      (parênteses forçam a prioridade)
//          5 * 4 = 20      (multiplicação)
//          10 / 5 = 2      (divisão)
//          20 - 2          (subtração por último)
//          Resultado: 18
```

Sem os parênteses, a mesma expressão $2 + 3 * 4 - 10 / 5$ resultaria em 10 , pois $3 * 4$ seria resolvido primeiro. O simples uso de parênteses alterou o resultado de 10 para 18 . O programa abaixo aplica esse mesmo raciocínio com um conjunto maior de variáveis e imprime o resultado de três expressões distintas.

Exemplo 2: Precedência na prática

```
%%writefile cpp/src/02_exec_02.cpp
#include <iostream>

using namespace std;

int main(){

    int a {6};
    int b {3};
    int c {8};
    int d {9};
    int e {3};
    int f {2};
    int g {5};

    int result = a + b * c - d / e - f + g;
    // 6 + 3 * 8 - 9 / 3 - 2 + 5 = 6 + 24 - 3 - 2 + 5 = 30
    cout << "Resultado: " << result << endl;

    result = a / b * c + d - e + f;
    // 6 / 3 * 8 + 9 - 3 + 2 = 2 * 8 + 9 - 3 + 2 = 16 + 9 - 3 + 2 = 24
    cout << "Resultado: " << result << endl;

    result = (a + b) * c - d / e - f + g;
    // (6 + 3) * 8 - 9 / 3 - 2 + 5 = 9 * 8 - 3 - 2 + 5 = 72 - 3 - 2 + 5 = 72
    cout << "Resultado: " << result << endl;

    result = b % f * f;
    // 3 % 2 * 2 = 1 * 2 = 2
    cout << "Resultado: " << result << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_02`, ou utilize a célula abaixo no próprio Jupyter Notebook para

facilitar.

```
!./cpp/bin/02_exec_02
```

2.3 Incremento e Decremento

O C++ oferece dois operadores especiais para somar ou subtrair 1 de uma variável. O operador `++` incrementa o valor em 1 e o operador `--` decrementa o valor em 1. Na prática, eles substituem a forma manual que já conhecemos.

```
int value {5};
value = value + 1; // forma manual, value agora vale 6
value = value - 1; // forma manual, value volta a valer 5
```

Os operadores `++` e `--` fazem exatamente isso, mas de forma mais compacta. A diferença é que eles podem ser posicionados antes da variável (prefixo) ou depois da variável (posfixo), e essa posição muda o momento em que o incremento acontece.

Forma	Sintaxe	Comportamento
Prefixo	<code>++value</code>	Incrementa primeiro, depois usa o novo valor
Posfixo	<code>value++</code>	Usa o valor atual primeiro, depois incrementa

Uma analogia simples para lembrar a diferença. No modo prefixo, você paga antes de usar, ou seja, o valor já chega atualizado. No modo posfixo, você usa agora e paga depois, ou seja, o valor antigo é utilizado e só então a variável é atualizada.

Para ver essa diferença na prática, considere o operador prefixo dentro de um `cout`.

```
int value {5};
cout << ++value << endl;
// O valor é incrementado para 6 antes de ser impresso
// Saída: 6
```

Agora compare com o operador posfixo na mesma situação.

```
int value {5};
cout << value++ << endl;
// O valor 5 é impresso primeiro, depois é incrementado para 6
// Saída: 5
cout << value << endl;
// Agora o incremento já aconteceu
// Saída: 6
```

Perceba que no modo posfixo, o `cout` recebeu o valor antigo (5) e somente depois a variável foi atualizada para 6. Essa distinção é a essência da diferença entre as duas formas.

Quando o operador aparece sozinho em uma linha, separado de qualquer outra expressão, ambas as formas produzem o mesmo resultado final.

```
int value {5};
++value; // prefixo, value agora vale 6
cout << value << endl;
// Saída: 6

int value2 {5};
value2++; // posfixo, value2 agora vale 6
cout << value2 << endl;
// Saída: 6
```

Nos dois casos o valor final é 6. A diferença entre prefixo e posfixo só se manifesta quando o operador está dentro de uma expressão que utiliza o valor no mesmo momento, como no argumento do `cout`. Quando o operador aparece isolado na linha, não há expressão consumindo o valor intermediário, então o efeito é idêntico. O programa abaixo demonstra cada um desses cenários em sequência.

Exemplo 3: Incremento e decremento na prática

```
%writefile cpp/src/02_exec_03.cpp
#include <iostream>

using namespace std;

int main(){

    int value {5};
```

```

    cout << "0 valor inicial: " << value << endl;

    cout << endl;

    // Incrementa 1

    value = value + 1;
    cout << "Incremento - valor é: " << value << endl;

    value = 5; // Resetando para 5

    // Decrementando 1

    value = value - 1;
    cout << "Decremento - valor é: " << value << endl;

    //=====

    cout << endl;

    cout << "Posfixo - Incremento e Decremento" << endl;

    value = 5; // Resetando para 5
    cout << "0 valor inicial resetado: " << value << endl;

    // Incremento de 1
    cout << "0 valor é (posfixo++): " << value++ << endl;
    cout << "0 valor é: " << value << endl;

    value = 5; // Resetando para 5

    // Decremento de 1

    cout << "0 valor é (posfixo--): " << value-- << endl;
    cout << "0 valor é: " << value << endl;

    //=====

    cout << endl;

    cout << "Prefixo - Incremento e Decremento" << endl;

    value = 5; // Resetando para 5
    cout << "0 valor inicial resetado: " << value << endl;

    // Incremento de 1

    cout << "0 valor é (++prefixo): " << ++value << endl;

    value = 5; // Resetando para 5
    cout << "0 valor inicial resetado: " << value << endl;

    // Decremento de 1

    cout << "0 valor é (--prefixo): " << --value << endl;

    return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_03.cpp -o cpp/bin/02_exec_03
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_03`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/02_exec_03
```

2.3. Operadores Compostos

Os operadores compostos combinam uma operação aritmética com a atribuição em um único passo. Eles existem como atalho sintático para tornar o código mais enxuto quando queremos modificar o valor de uma variável com base em seu próprio valor atual. Considere a seguinte operação.

```
int score {10};
score = score + 3; // forma expandida, score agora vale 13
```

Com o operador composto, a mesma operação é escrita de forma mais direta.

```
int score {10};
score += 3; // forma composta, score agora vale 13
```

Não existe diferença de comportamento entre as duas formas. O resultado é idêntico. A vantagem é puramente de legibilidade e economia de digitação. A tabela abaixo resume todos os operadores compostos disponíveis.

Operador	Operação	Exemplo	Equivalente Expandido
<code>+=</code>	Adição e atribuição	<code>value += 5</code>	<code>value = value + 5</code>
<code>-=</code>	Subtração e atribuição	<code>value -= 5</code>	<code>value = value - 5</code>
<code>*=</code>	Multiplicação e atribuição	<code>value *= 2</code>	<code>value = value * 2</code>
<code>/=</code>	Divisão e atribuição	<code>value /= 3</code>	<code>value = value / 3</code>
<code>%=</code>	Módulo e atribuição	<code>value %= 11</code>	<code>value = value % 11</code>

Um detalhe importante é que cada operação composta modifica a própria variável. O valor não é armazenado em outro lugar, ele é atualizado diretamente. Na prática, isso significa que o resultado de uma operação vira o ponto de partida da próxima. Acompanhe a sequência abaixo.

```
int total {100};
total -= 20; // 100 - 20 = 80
total *= 3; // 80 * 3 = 240
total /= 4; // 240 / 4 = 60
total %= 7; // 60 % 7 = 4 (60 dividido por 7 é 8 com resto 4)
```

Observe como cada linha parte do resultado anterior. O `total` começa em 100, vira 80 após a subtração, salta para 240 com a multiplicação, cai para 60 na divisão e termina em 4 com o módulo. Se trocarmos a ordem de qualquer operação, o resultado final muda completamente, pois o valor de entrada de cada passo depende do passo anterior. O programa abaixo aplica essa mesma lógica e imprime o valor da variável após cada operação composta.

Exemplo 4. Operadores compostos em sequência

```
%%writefile cpp/src/02_exec_04.cpp
#include <iostream>

using namespace std;

int main(){

    int value {45};

    cout << "O valor é: " << value << endl;
    cout << endl;

    // Adição
    value = value + 5;
    cout << "O valor é (depois +=5) : " << value << endl;
    cout << endl;

    // Subtração
    value -= 5;
    cout << "O valor é (depois -=5) : " << value << endl;
    cout << endl;

    // Multiplicação
    value *= 2;
    cout << "O valor é (depois *=2) : " << value << endl;
    cout << endl;

    // Divisão
    value /= 3;
    cout << "O valor é (depois /=3) : " << value << endl;
    cout << endl;

    // Resto
    value %= 11;
    cout << "O valor é (depois %=11) : " << value << endl;
    cout << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_04.cpp -o cpp/bin/02_exec_04
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_04`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/02_exec_04
```

2.4. Operadores de Comparação

Os operadores de comparação, também chamados de operadores relacionais, comparam dois valores e retornam um resultado do tipo `bool`, ou seja, `true` (verdadeiro) ou `false` (falso).

Operador	Significado	Exemplo	Resultado
<code>==</code>	Igual a	<code>45 == 60</code>	<code>false</code>
<code>!=</code>	Diferente de	<code>45 != 60</code>	<code>true</code>
<code>></code>	Maior que	<code>45 > 60</code>	<code>false</code>
<code><</code>	Menor que	<code>45 < 60</code>	<code>true</code>
<code>>=</code>	Maior ou igual a	<code>45 >= 60</code>	<code>false</code>
<code><=</code>	Menor ou igual a	<code>45 <= 60</code>	<code>true</code>

Um erro muito comum entre iniciantes é confundir o operador de atribuição `=` com o operador de igualdade `==`. Os dois possuem significados completamente diferentes.

```
int x {10};
x = 20;           // atribuição, copia o valor 20 para x
bool r = (x == 20); // comparação, verifica se x é igual a 20 e retorna true
```

O operador `=` copia um valor para uma variável. O operador `==` compara dois valores e retorna `true` ou `false`. Trocar um pelo outro é uma fonte frequente de bugs.

Observe no exemplo acima que o resultado da comparação `(x == 20)` foi armazenado em uma variável do tipo `bool`. Isso é possível porque toda comparação produz um valor concreto, `true` ou `false`, que pode ser guardado e reutilizado em outras partes do programa.

```
int temperatura {38};
bool febre = (temperatura > 37);
// 38 > 37 é true, então febre recebe true
```

Por padrão, o `cout` exibe valores booleanos como números, `1` para `true` e `0` para `false`. Para tornar a saída mais legível, utilizamos o manipulador `boolalpha` antes da impressão.

```
bool ativo {true};
cout << ativo << endl;           // Saída: 1
cout << boolalpha;
cout << ativo << endl;           // Saída: true
```

Após inserir `boolalpha`, todas as impressões seguintes de valores booleanos exibirão as palavras `true` e `false` em vez de `1` e `0`. O programa abaixo aplica todos os operadores relacionais sobre duas variáveis e imprime cada resultado com `boolalpha` ativado.

Exemplo 5. Comparações entre variáveis

```
%writefile cpp/src/02_exec_05.cpp
#include <iostream>

using namespace std;

int main(){

    int number1 {45};
    int number2 {60};

    cout << "number1: " << number1 << endl;
    cout << "number2: " << number2 << endl;
```

```

cout << endl;

cout << "Comparando variáveis: " << endl;

cout << boolalpha;

cout << "number1 < number2: " << (number1 < number2) << endl;
cout << "number1 <= number2: " << (number1 <= number2) << endl;
cout << "number1 > number2: " << (number1 > number2) << endl;
cout << "number1 >= number2: " << (number1 >= number2) << endl;
cout << "number1 == number2: " << (number1 == number2) << endl;
cout << "number1 != number2: " << (number1 != number2) << endl;

cout << endl;

cout << "Armazene e depois compare" << endl;

bool result = (number1 == number2);

cout << number1 << " == " << number2 << " : " << result << endl;

return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_05.cpp -o cpp/bin/02_exec_05
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_05`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/02_exec_05
```

2.5. Operadores Lógicos

Os operadores lógicos permitem combinar múltiplas expressões booleanas em uma única expressão. Eles operam sobre valores `true` e `false` e retornam um resultado também booleano.

Operador	Nome	Regra
<code>&&</code>	AND (E lógico)	Resulta em <code>true</code> somente quando todos os operandos forem <code>true</code> . Um único <code>false</code> torna o resultado <code>false</code>
<code> </code>	OR (OU lógico)	Resulta em <code>true</code> quando pelo menos um operando for <code>true</code> . Um único <code>true</code> já torna o resultado <code>true</code>
<code>!</code>	NOT (negação)	Inverte o valor. <code>!true</code> se torna <code>false</code> e <code>!false</code> se torna <code>true</code>

Para fixar o comportamento de cada operador, as tabelas verdade abaixo mostram todos os resultados possíveis.

AND (`&&`). Ambos os operandos precisam ser verdadeiros para o resultado ser verdadeiro.

A	B	A && B
<code>true</code>	<code>true</code>	<code>true</code>
<code>true</code>	<code>false</code>	<code>false</code>
<code>false</code>	<code>true</code>	<code>false</code>
<code>false</code>	<code>false</code>	<code>false</code>

OR (`||`). Basta um operando verdadeiro para o resultado ser verdadeiro.

A	B	A B
<code>true</code>	<code>true</code>	<code>true</code>
<code>true</code>	<code>false</code>	<code>true</code>
<code>false</code>	<code>true</code>	<code>true</code>
<code>false</code>	<code>false</code>	<code>false</code>

NOT (`!`). Simplesmente inverte o valor recebido.

A	!A
<code>true</code>	<code>false</code>
<code>false</code>	<code>true</code>

Veja como isso funciona na prática com variáveis booleanas.

```

bool x {true};
bool y {false};

bool r1 = (x && y); // true && false = false

```



```
bool r2 = (x || y); // true || false = true
bool r3 = !x;      // !true = false
```

Na primeira linha, o `&&` exige que ambos sejam verdadeiros, mas `y` é `false`, então o resultado é `false`. Na segunda, o `||` precisa de apenas um verdadeiro, e `x` já é `true`, então o resultado é `true`. Na terceira, o `!` simplesmente inverte `true` para `false`.

Esses operadores também podem ser combinados entre si para formar expressões mais elaboradas.

```
bool x {true};
bool y {false};
bool z {true};

bool r = !(x && y) || z;
// Passo 1: x && y = true && false = false
// Passo 2: !(false) = true
// Passo 3: true || z = true || true = true
```

Avaliação em curto circuito: O compilador possui uma otimização importante ao avaliar expressões lógicas. No operador `&&`, quando o primeiro operando é `false`, o compilador já sabe que o resultado será `false` independentemente do segundo operando, então ele nem avalia o segundo. No operador `||`, quando o primeiro operando é `true`, o compilador já sabe que o resultado será `true` e pula a avaliação do segundo.

```
bool p {false};
bool q {true};

bool r = (p && q);
// p é false, o compilador já sabe que o resultado é false
// q nem é avaliado
```

Combinando operadores lógicos com operadores relacionais. Na prática, raramente comparamos variáveis `bool` diretamente. O uso mais frequente dos operadores lógicos é combinar comparações entre valores numéricos. Cada comparação produz um `bool`, e o operador lógico combina esses resultados.

```
int nota {75};
int faltas {3};

bool aprovado = (nota >= 60) && (faltas <= 5);
// Passo 1: 75 >= 60 = true
// Passo 2: 3 <= 5 = true
// Passo 3: true && true = true
```

Agora considere uma situação onde uma das condições falha.

```
int nota {75};
int faltas {3};

bool reprovado = (nota >= 60) && (nota <= 40);
// Passo 1: 75 >= 60 = true
// Passo 2: 75 <= 40 = false
// Passo 3: true && false = false
```

A primeira comparação resulta em `true`, mas a segunda resulta em `false`. Como o `&&` exige que ambas sejam verdadeiras, o resultado final é `false`. O programa abaixo demonstra todos esses operadores em sequência, primeiro com variáveis booleanas puras, depois combinando operadores lógicos com relacionais sobre variáveis inteiras.

Exemplo 6. Operadores lógicos e combinação com relacionais

```
%%writefile cpp/src/02_exec_06.cpp
#include <iostream>

using namespace std;

int main(){

    bool a {true};
    bool b {false};
    bool c {true};
```

```

cout << boolalpha;

/*
AND :
    0 resultado é verdadeiro quando todos os operandos são verdadeiros.
    Um único operando falso fará com que toda a expressão seja avaliada como falsa.
*/

cout << "a: " << a << endl;
cout << "b: " << b << endl;
cout << "c: " << c << endl;

cout << endl;
cout << "Operação AND" << endl;

cout << " a && b : " << (a && b) << endl;
cout << " a && c : " << (a && c) << endl;
cout << " a && b && c : " << (a && b && c) << endl;

/*
OR :
    Avalia como verdadeiro quando pelo menos um operando é verdadeiro.
    Um único operando verdadeiro fará com que toda a expressão seja avaliada como verdadeira.
*/

cout << endl;
cout << "Operação OR" << endl;

cout << " a || b : " << (a || b) << endl;
cout << " a || c : " << (a || c) << endl;
cout << " a || b || c : " << (a || b || c) << endl;

//NOT : Nega qualquer operando com o qual você o tenha combinado.

cout << endl;
cout << "Operação NOT" << endl;

cout << "!a : " << !a << endl;
cout << "!b : " << !b << endl;
cout << "!c : " << !c << endl;

cout << endl;
cout << "Combinando Operações" << endl;

cout << "!(a &&b) || c : " << (!(a &&b) || c) << endl; // true

// Combinando Operadores Lógicos com Operadores Relacionais

int d {45};
int e {20};
int f {11};

cout << endl;
cout << "Operações relacionais e lógicas com números inteiros" << endl;
cout << "d : " << d << endl;
cout << "e : " << e << endl;
cout << "f : " << f << endl;

cout << endl;

cout << "(d > e) && (d > f) : " << ((d > e) && (d > f)) << endl; // True
cout << "(d == e) || (e <= f) : " << ((d==e) || (e <= f)) << endl; // False
cout << "(d < e) || (d > f) : " << ((d < e) || (d > f)) << endl;
cout << "(f > e) || (d < f) : " << ((f > e) || (d < f)) << endl;
cout << "(d > f) && (f <= d) : " << ((d > f) && (f <= d)) << endl;
cout << "(d > e) && (d <= f) : " << ((d > e) && (d <= f)) << endl;
cout << "(! a) && (d == e) : " << ((! a) && (d == e)) << endl;
cout << "(! a) && (d == e) : " << ((! a) && (d == e)) << endl;

return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/02_exec_06.cpp -o cpp/bin/02_exec_06
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/02_exec_06`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/02_exec_06
```

2.6 Resumo dos Operadores

A tabela abaixo reúne todos os operadores apresentados neste módulo, organizados por categoria.

Categoria	Operadores
Atribuição	=
Aritméticos	+ - * / %
Incremento e Decremento	++ --
Compostos	+= -= *= /= %=
Relacionais	== != > < >= <=
Lógicos	&& !

Pontos importantes para lembrar

1. A divisão entre dois `int` sempre descarta a parte decimal. Para obter resultado fracionário, pelo menos um dos operandos deve ser `double` ou `float`.
2. Multiplicação, divisão e módulo possuem prioridade maior que adição e subtração. Operadores de mesma prioridade são resolvidos da esquerda para a direita. Parênteses `()` forçam qualquer ordem desejada.
3. Os operadores `++` e `--` no modo prefixo atualizam o valor antes de usá-lo na expressão. No modo posfixo, o valor antigo é usado primeiro e a atualização acontece depois. Quando aparecem sozinhos na linha, ambas as formas produzem o mesmo resultado.
4. Os operadores compostos (`+=`, `-=`, `*=`, `/=`, `%=`) são atalhos que modificam a variável diretamente, sem criar variáveis auxiliares. Cada operação parte do resultado anterior.
5. Toda comparação relacional produz um valor `bool` (`true` ou `false`) que pode ser armazenado em uma variável para uso posterior. O manipulador `boolalpha` faz o `cout` exibir `true` e `false` em vez de `1` e `0`.
6. O operador `=` atribui um valor. O operador `==` compara dois valores. Confundir os dois é um dos erros mais comuns entre iniciantes.
7. O operador `&&` exige que todos os operandos sejam verdadeiros. O operador `||` exige que pelo menos um seja verdadeiro. O operador `!` inverte o valor recebido.
8. Na avaliação em curto circuito, o compilador pula o segundo operando quando o primeiro já determina o resultado. No `&&`, um primeiro operando `false` encerra a avaliação. No `||`, um primeiro operando `true` encerra a avaliação.